

Description
Accepted ✕
Editorial
Solutions
Submissions

All Submissions

Accepted 89 / 89 testcases passed

yousab21 submitted at Apr 14, 2026 18:57

Analysis
Solution

Runtime
0 ms | Beats 100.00% 🏆

Memory
11.47 MB | Beats 97.48% 🏆

Code | C++

```

1 //here the obviose anser is to just implement an inplace sorting standard al
2 //that would be boring, i also notices that the problem have a follow up to
3 //so i tried doing that, and while i didnt find a solotion with one pass i c
4 class Solution {
5 public:
6     void sortColors(vector<int>& nums) {
7         //since each element has just 3 posibilities we dont need to sort the
8         //we can just count how many of each color exsist in the array and j

```

View more

More challenges

280. Wiggle Sort
324. Wiggle Sort II

Write your notes here

Select related tags 0/5

Code

C++ Auto

```

27 //here the obviose anser is to just implement an inplace sorting standard algorithm like selection or bubble sort
26 //that would be boring, i also notices that the problem have a follow up to find a solution in one pass
25 //so i tried doing that, and while i didnt find a solotion with one pass i came up with one that does it in 2
24 class Solution {
23 public:
22     void sortColors(vector<int>& nums) {
21         //since each element has just 3 posibilities we dont need to sort the array with normale comparasions and swaps
20         //we can just count how many of each color exsist in the array and just reright the array to be sorted with those numbers
19         //this is the first pass i just count the colors with a simple switch statment
18         int numOfZeros = 0;
17         int numOfOnes = 0;
16         int numOfTwos = 0;
15         for (int i = 0 ; i < nums.size() ; i++){
14             switch(nums[i]){
13                 case 0:
12                     numOfZeros++;
11                     break;
10                 case 1:
9                     numOfOnes++;
8                     break;
7                 case 2:
6                     numOfTwos++;
5                     break;
4             }
3         }
2
1         //here is the second pass i just walk the array and rewrite it to be sorted
28 //with the exact color numbers i counted in the previos pass
1         int index = 0;
2         while(numOfZeros){
3             nums[index] = 0;
4             index++;
5             numOfZeros--;
6         }
7         while(numOfOnes){
8             nums[index] = 1;
9             index++;
10            numOfOnes--;
11        }
12        while(numOfTwos){
13            nums[index] = 2;
14            index++;
15            numOfTwos--;
16        }
17    }
18 };

```

Saved

Ln 28, Col 69

Testcase
Test Result

Sorting

DescriptionAcceptedEditorialSolutionsSubmissions

All Submissions

Accepted59 / 59 testcases passed

yousab21 submitted at Apr 14, 2026 19:27

AnalysisSolution

Runtime

0 msBeats 100.00%

Memory

12.22 MBBeats 71.71%

Time Interval	Performance (%)
1ms	100%
2ms	0%
3ms	0%
4ms	0%

CodeC++

```
1 //this is just the standard merge algorithm used with merge sort
2 //just insted of returning a the new array we pass it by refrence to nums1
3
4 class Solution {
5 public:
6     void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
7         vector<int> newArray;
8         int i = 0 , j = 0;
```

View more

More challenges

977. Squares of a Sorted Array

986. Interval List Intersections

2516. Take K of Each Character From Left and Right

Write your notes here

Select related tags0/5

Code

C++Auto

```
15 //this is just the standard merge algorithm used with merge sort
14 //just insted of returning a the new array we pass it by refrence to nums1
13
12 class Solution {
11 public:
10     void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
9         vector<int> newArray;
8         int i = 0 , j = 0;
7         //here i just compare elements from both array and push the smaller one
6         //into the new array until i reach the end of any of the arrays
5         while(i < m && j < n){
4             if(nums1[i] < nums2[j]){
3                 newArray.push_back(nums1[i]);
2                 i++;
1             }
16            else{
1             newArray.push_back(nums2[j]);
2             j++;
3             }
4         }
5         //here i just copy the remaining elements in the other array
6         while(i < m){
7             newArray.push_back(nums1[i]);
8             i++;
9         }
10        while(j < n){
11            newArray.push_back(nums2[j]);
12            j++;
13        }
14        //then i just pass the new array to nums 1 by refrence
15        nums1 = newArray;
16    }
17 };
```

SavedLn 16, Col 17

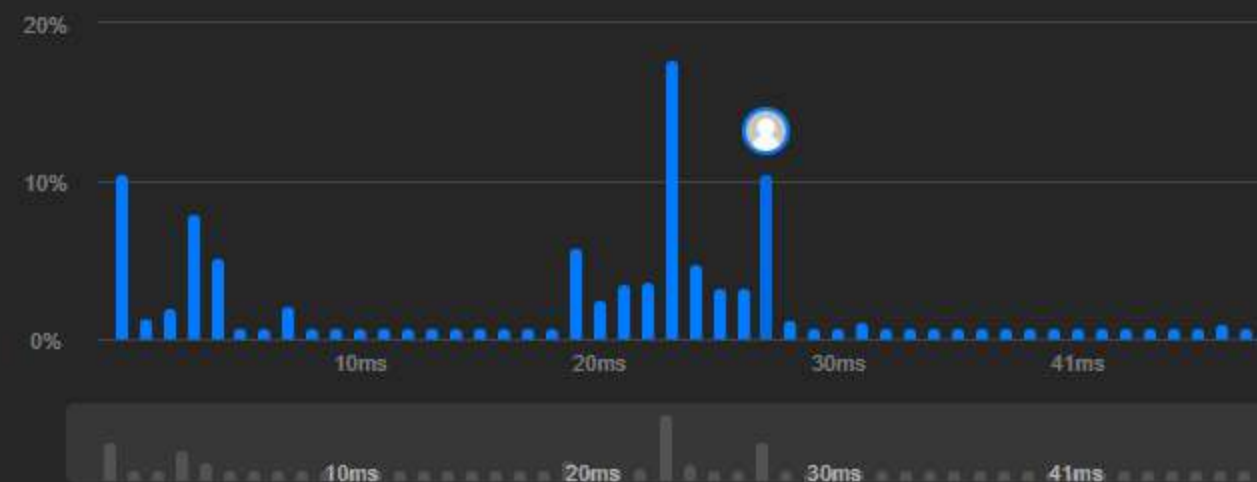
Testcase

Test Result

 Solution

 **Memory**

14.43 MB Beats 97.31% 🏆



View more

- **708. Insert into a Sorted Circular Linked List**

0/5

C++   Auto

```

38 class Solution {
39 public:
40     ListNode* insertionSortList(ListNode* head) {
41         //regular edge case if list is empty or have one node
42         if(!head || !head->next){
43             return head;
44         }
45
46         //if the head is not the smallest node smaller nodes will be inserted
47         //before it so i make a dummy to simplify returns
48         ListNode* dummy = new ListNode(0,head);
49
50         //ill use key and before key to to traverse the list in the outer loop and
51         //current and prev to do the comparasions in the inner loop
52         ListNode* key = head;
53         ListNode* beforeKey = dummy;
54         ListNode* prev = dummy;
55         ListNode* current = dummy->next;
56
57         while(key){
58             //here i reset the inner loop pointers
59             prev = dummy;
60             current = dummy->next;
61
62             //here i advance prev and current until we fine a value bigger than key
63             //or we reach the key
64             while( current != key && current->val < key->val ){
65                 prev = current;
66                 current = current->next;
67             }
68
69             //if we reached the key this means it is already sorted as there is no larger elements before it
70             //so we do nothing and just advance to the next iteration
71             if(current == key){
72                 beforeKey= key;
73                 key = key->next;
74             }
75
76             //if we didnt reach the key however this means there is a number bigger than it in the
77             //sorted subarray so we do this rewiring to insert the key after said number
78             else{
79                 beforeKey->next = key->next;
80                 key->next = current;
81                 prev->next = key;
82                 key = beforeKey->next;
83             }
84         }
85         return dummy->next;
86     }
87 };

```

Saved

Ln 49, Col 89

Testcase > Test Result

Problem List

<

>

🔗

Description

Accepted

Editorial

Solutions

Submissions

All Submissions

🔗

Accepted

21 / 21 testcases passed

Analysis

Solution

👤

yousab21

submitted at Apr 15, 2026 21:10

🕒 Runtime

2639 ms | Beats 5.01%

💾 Memory

73.98 MB | Beats 58.34%

Code

C++

```
1 //here i need to implement quickSort however
2 // i think the guy in the material videos had his partition function be O(n^
3 // this will fail the limitaions of this problem so i found this partion alg
4 //i will explain how it works as i code it
5
6 //=====> PS: after submitting this failes a specific case in leet code desi
7 // the case is an arr full of large number with last element being 1
8 // for this i randomize reandomize the last index before partitioning to bre
```

View more

More challenges

2273. Find Resultant Array After Removing Anagrams

1980. Find Unique Binary String

3016. Minimum Number of Pushes to Type Word II

Write your notes here

Select related tags0/5

Code

Submit

Ctrl

Enter

C++

Auto

🔗

🔗

🔗

🔗

🔗

```
17 //here i need to implement quickSort however
16 // i think the guy in the material videos had his partition function be O(n^2) propably to simplify the algorithm
15 // this will fail the limitaions of this problem so i found this partion algorithm called (Lomuto partition) with O(n)
14 //i will explain how it works as i code it
13
12 //=====> PS: after submitting this failes a specific case in leet code designed to give it a bad pivot
11 // the case is an arr full of large number with last element being 1
10 // for this i randomize reandomize the last index before partitioning to break the evil case
9
8 //=====> PS again: there is another evil case
7 //this time it is a very big array will all 2s and one 3 so the pivot is always bad
6 //the solotion this time is to completly scrap lomuto and use an algorithm called the "Dutch National Flag partition"
5 //i will explain how it works as i write it (to the best as i understand it)
4
3 //(i dont know why someone would submit cases like this)
2 //=====[ AND NO I DID NOT JUST COME UP WITH THESE SOLUTION ON MY OWN ]
1
18 class Solution {
19 public:
20     //the idea here is insted of working with a single pivot index we work with a pivot region where it houses
21     // all equal elemets to the pivot and we return a pair with the start and end of this pivot region
22     //while partitioning we seperate the array into 4 regions like this :
23     // [ < pivot , =pivot , unknown , > povot ]
24     //and we proccess everything in the unkown region adding each element to its correct region
25     //the goal is to eleminate thje unkown region and end up with 3 regions [< pivot , =pivot , > pivot]
26     pair<int,int> partition(vector<int>& arr, int low, int high) {
27
28         int current = low;
29         int pivotRegioValue = arr[high];
30
31         //these here will mark the 3 result regions
32         int L_B_P = low;    //last element before pivot region
33         int F_A_P = high;   //first element after the pivot region
34
35         //we loop from our curent position
36         while (current ≤ F_A_P) {
37             //if the value is bigger than the pivot we add it to the [> pivot region]
38             if (arr[current] < pivotRegioValue) {
39                 swap(arr[L_B_P], arr[current]);
40                 L_B_P++;
41                 current++;
42             }
43             //if it is smaller we add it to the < pivot region
44             else if (arr[current] > pivotRegioValue) {
45                 swap(arr[current], arr[F_A_P]);
46                 F_A_P--;
47             }
48             //if the value = the pivot we dont need to move it as by moving smaller and bigger value
49             //we naturally end up with the = pivot region in the middle (the unkown region is now known to be = pivot)
```

Saved

Ln 18, Col 17

Testcase

Test Result

 **Solution**

73.98 MB | Beats 58.34% 🟢



View more

0/5

Testcase > Test Result

```

29 // this is just standard heapSortc pretty much exactly ass the material video
28 #include <algorithm>
27 #include <iostream>
26 #include <vector>
25
24 // this is the heapify function the goal of this step is to rearrange
23 // the imajanary tree passed to it so that the root is bigger than its children
22 // following the max heap rules
21 void heapify(std::vector<int> &arr, int root, int size) {
20     // the way we map the array elements to the tree is by multiplaing the root by
19     // 2 and adding 1 and 2 to get its left and right children
18     int left = 2 * root + 1;
17     int right = 2 * root + 2;
16
15     // we begin be assuming the root is the max value
14     int max = root;
13
12     // and then we compare it with its left and right child and track the max
11     // between the three also it is important to pass to check if passed
10     // the array boundry (totally didnt get a seg fault here)
9     if (left < size && arr[left] > arr[max]) {
8         max = left;
7     }
6     if (right < size && arr[right] > arr[max]) {
5         max = right;
4     }
3
2     // if max value was so longer the root we swap it with the child who is now
1     // max and continue to heapfity down the tree
30 if (max != root) {
1     std::swap(&a: arr[max], &b: arr[root]);
2     heapify(&: arr, root: max, size);
3 }
4

```

```

→ yousab ~ cd Workspaces/CatReloaded-tasks/week
5
→ yousab week5 g++ heapSort.cpp -o test
→ yousab week5 ./test
Sorted array: 5 6 7 11 12 13
→ yousab week5
→ yousab week5

```



```

4      std::swap(&a: arr[max], &b: arr[root]);
3      heapify(&: arr, root: max, size);
2  }
1
35 // if the root is alreadyt the max value we are done and we can return
1  }
2
3  // this where i actually build the max heap by calling heapify on all posiible
4  // roots nodes starting from the bottom doing i = array.size()/2 - 1 gets us the
5  // last element that have children as the rest are all bottom level leafs the
6  // reson this is true is be because those elements can be written as {some
7  // element*2 + 1} and {some element*2 + 2} which is exacly our difintion of left
8  // and right child in the heapify function
9  void buildMaxHeap(std::vector<int> &arr) {
10     for (int i = arr.size() / 2 - 1; i ≥ 0; i--) {
11         heapify(&: arr, root: i, size: arr.size());
12     }
13 }
14
15 // this is the main heaoSort funciiion wehere we first build the full max heap
16 // and since the root is the max value swapping it with the last element in the
17 // array puts it in its correct place but this messes up the heap so we call
18 // heapify on the new root to resotore max heap rules we keep doing this until
19 // we have iterated over the whole array and sorted it
20 void heapSort(std::vector<int> &arr) {
21     buildMaxHeap(&: arr);
22
23     for (int i = arr.size() - 1; i > 0; i--) {
24         std::swap(&a: arr[0], &b: arr[i]);
25         heapify(&: arr, root: 0, size: i);
26     }
27 }
28
29 // this is just some example code to test heapSort

```

```

→ yousab ~ cd Workspaces/CatReloaded-tasks/week
5
→ yousab week5 g++ heapSort.cpp -o test
→ yousab week5 ./test
Sorted array: 5 6 7 11 12 13
→ yousab week5
→ yousab week5

```

```
29 // array puts it in its correct place but this messes up the heap so we call
28 // heapify on the new root to restore max heap rules we keep doing this until
27 // we have iterated over the whole array and sorted it
26 void heapSort(std::vector<int> &arr) {
25     buildMaxHeap(& arr);
24
23     for (int i = arr.size() - 1; i > 0; i--) {
22         std::swap(&a: arr[0], &b: arr[i]);
21         heapify(& arr, root: 0, size: i);
20     }
19 }
18
17 // this is just some example code to test heapSort
16 int main() {
15     std::vector<int> arr = {[0]=12, [1]=11, [2]=13, [3]=5, [4]=6, [5]=7};
14     heapSort(& arr);
13
12     std::cout << "Sorted array: ";
11     for (int i = 0; i < arr.size(); i++) {
10         std::cout << arr[i] << " ";
9     }
8     std::cout << std::endl;
7     return 0;
6 }
5
4 // also how do you like my ricing XD
3 // i use NeoVim with lazyVim as my editor
2 // catppuccin mocha colorscheme
1 // for terminal i like kitty
81 // L O N G     L I V E     L I N U X
```

```
→ yousab ~ cd Workspaces/CatReloaded-tasks/week
5
→ yousab week5 g++ heapSort.cpp -o test
→ yousab week5 ./test
Sorted array: 5 6 7 11 12 13
→ yousab week5
→ yousab week5
```


Search...

Courses

Tutorials

Interview Prep

Problem

Editorial

Submissions

Comments

Output Window

Compilation Results

Custom Input

Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

1115 / 1115

Attempts : Correct / Total

1 / 1

Accuracy : 100%

Points Scored

2 / 2

Your Total Score: 6

Time Taken

0.15

Solve Next

Selection Sort

Insertion Sort

Counting Sort

Stay Ahead With:

Build 21 Projects in 21 Days

Build real-world ML, Deep Learning & Gen AI projects

Register Now

C++ (17)

Start Timer

```
1 //this is just standard bubble sort i even gave it some brains with an early exit flag
2 class Solution {
3     public:
4     void bubbleSort(vector<int>& arr) {
5         //you loop over array elements
6         for(int i = 0 ; i < arr.size() ; i++){
7             //if we completed an iteration without doing any swaps this meand than
8             //that every element is smaller than the elemtn after it aka the array is sorted
9             bool didSwaps = false;
10
11             //you move large elements to the end of the array
12             for (int j = 0 ; j < arr.size() - i - 1 ; j++ ){
13                 if(arr[j] > arr[j+1]){
14                     swap(arr[j], arr[j+1]);
15                     didSwaps = true;
16                 }
17             }
18
19             //if the array is sorted we just exit
20             if(!didSwaps){
21                 return;
22             }
23         }
24     }
25 }
26 ;|
```

Custom Input

Compile & Run

Submit